ADAM (A Decentralized Approach Model) İşbirlikçi Konsensüsü

1. Giriş ve Arka Plan

ADAM (A Decentralized Approach Model) konsensüs mekanizması; blok zincirini madencilik merkezileşmesine, bencil madenciliğe (selfish mining), blok grinding işlemlerine ve blok liderlerine yönelik hedefli Denial-of-Service (DoS) saldırılarına karşı güçlendirmek için tasarlanmış işbirlikçi bir hibrit konsensüs modelidir.

Geleneksel Proof-of-Work (PoW) ve Proof-of-Stake (PoS) protokolleri belirgin güvenlik açıklarına sahiptir:

- PoW modelinde, devasa işlem (hashing) gücüne sahip madenciler blok üretimine hakim olur ve bu durum konsensüsü merkezileştiren havuzların oluşmasına yol açar.
- PoS modelinde, en zengin stake eden cüzdanların blok üretme olasılığı orantısız şekilde daha yüksektir.
- Her iki modelde de, bir sonraki blok üreticisinin kimliği önceden biliniyorsa (önceden seçilmişse), DoS saldırılarının veya rüşvet girişimlerinin hedefi haline gelirler. Eğer blok içerikleri bir sonraki bloğun rastgeleliğini değiştirecek şekilde manipüle edilebiliyorsa, madenciler gelecekteki lider seçimlerini saptırmak için *grinding saldırıları* gerçekleştirebilirler.

ADAM, blok üretim sürecini bir **İşbirlikçi Hibrit Turuna** bölerek bu sorunları çözer:

1. **Seçim Aşaması (Election Phase)**: Her blok slotu için, sözde rastgele bir seed deterministik olarak N **adet Madenci (Miners)** ve **1 adet Koordinatör (Coordinator)** havuzunu seçer.
2. **Çözme Aşaması (Solving Phase)**: Seçilen Madenciler, mevcut bloğun hedefine bağlı olarak hafif, paralel hale getirilmiş PoW bulmacalarını (kısmi çözümler) derler ve çözer.
3. **Birleştirme Aşaması (Aggregation Phase)**: Seçilen Koordinatör kısmi çözümleri toplar, doğrular, bunları bir blok şablonu (block template) halinde paketler, kendi gizli anahtarını (private key) kullanarak nihai blok başlığını (block header) imzalar ve bunu ağa yayınlar.

---

2. Konsensüs Mimarisi ve Parametreleri

ADAM konsensüsü, blok yüksekliğine bağlı olarak koşullu bir şekilde etkinleştirilir. Temel parametreler `src/consensus/params.h` dosyasında tanımlanmıştır ve ağ bazında `src/chainparams.cpp` dosyasında yapılandırılmıştır.

Aktif Masternode sayısının düşük olduğu durumlarda ağın sorunsuz bir şekilde bootstrap (ilk kurulum) yapabilmesini sağlamak için ADAM, konsensüs yükseltme yükseklikleriyle deterministik olarak denetlenen iki blok sürümü moduna sahiptir.

A. Fallback Mode (Geri Çekilme Modu - Blok Versiyon 11)

Fallback Mode, ağın bootstrap aşaması (`UPGRADE_ADAM` $\leq H <$ `UPGRADE_POMBL`) için tasarlanmıştır. Aktif Masternode kaydı gerektirmeden çalışır ve tamamen kendi kendine yeten (self-contained) bir yapıdadır.

- **Madenci Sayısı (N)**: $11 \le K \le 14$ olmak üzere, $K-1$ boyutundaki anahtar havuzuna (key pool) göre dinamik olarak belirlenir (anahtarlar deterministiktir ve seçilen Koordinatör `vAdamMiners` dizisinin son elemanı olarak eklenir).
- **Konsensüs Eşiği (T)**: `GetAdamThreshold(nHeight)` tarafından yönetilir (`nAdamThresholdBootstrapLimit` altında `5`, sonrasında `nAdamThreshold` uyarınca Mainnet/Regtest üzerinde `7`, Testnet üzerinde `3`).
- **Kendi Kendine Yeten Başlık Düzeni**: `vAdamMiners`, tüm K açık anahtarlarını depolar (ilk $K-1$ seçilen madenciler, son anahtar koordinatördür). `vAdamSolutions` ise $K-1$ kısmi çözümü depolar.

B. Standart Mod (Blok Versiyon 12)

Standart Mod, 3-permütasyonlu bulmaca özetleme zinciri ile tam işbirlikçi Çoklu Kanıt Algoritması (PoMBL) konsensüs durumunu temsil eder.

- **Madenci Sayısı (N)**: `nAdamMinersCount` konsensüs parametresi aracılığıyla yapılandırılır (`src/consensus/params.h` dosyasında tanımlanmış ve `src/chainparams.cpp` içinde Mainnet/Testnet/Regtest için `11` olarak başlatılmıştır).
- **Konsensüs Eşiği (T)**: `GetAdamThreshold(nHeight)` aracılığıyla yönetilir (`nAdamThresholdBootstrapLimit` altında `5`, sonrasında Mainnet/Regtest için `7`, Testnet için `3`).
- **Seçilen Koordinatör**: Aktif Masternode sayısı eşik değerini aştığında Koordinatör aktif Masternode listesinden dinamik olarak seçilir (`SelectAdamNodes` Kural 1) ve 11 kişilik `vAdamMiners` listesinden ayrıdır; aksi durumda `SelectAdamNodes` Kural 2 kalan havuzdaki en yüksek sıralamalı adayı Koordinatör olarak seçer.

C. Deterministik Blok Yüksekliği Aktivasyonu (`Consensus::UPGRADE_POMBL`)

Fallback Mode (Versiyon 11) ile Standart Mod (Versiyon 12) arasındaki geçiş, idari bir Spork anahtarına bağımlı olmaksızın doğrudan `Consensus::UPGRADE_POMBL` (`src/chainparams.cpp` içindeki `vUpgrades[Consensus::UPGRADE_POMBL].nActivationHeight`) yüksekliği ile deterministik olarak denetlenir:

- **Aktivasyon Yükseklikleri**: Mainnet üzerinde Blok `2.000`, Testnet üzerinde Blok `400` ve Regtest üzerinde Blok `300`.
- **Davranış**:
  - `UPGRADE_ADAM <= nHeight < UPGRADE_POMBL` aralığında: Bloklar Fallback Mode kuralları altında **Versiyon 11** olarak üretilir ve doğrulanır.
  - `nHeight >= UPGRADE_POMBL` olduğunda: `ComputeBlockVersion` **Versiyon 12** bloklar üretir; hem `ContextualCheckBlockHeader` hem de `CheckBlock` fonksiyonları `nVersion < 12` olan tüm blokları `bad-version` hatası ile kesin olarak reddeder.

Ağ Yapılandırmaları ve Aktivasyon Yükseklikleri

İş birlikçi konsensüs çerçevesinin geçişi üç ana aktivasyon eşiğinden oluşur:

1. `Consensus::UPGRADE_ADAM`: ADAM Fallback Modunu (Blok Sürümü 11) etkinleştirir.
2. `Consensus::UPGRADE_POMBL`: ADAM Standart Modunu / Çoklu Kanıt Algoritmasını (Multi-Proof-Algorithm - Blok Sürümü 12) etkinleştirir.
3. `Consensus::UPGRADE_MODELD`: Model D ödül dağılımlarını ve PoBLS konsensüsünü zorunlu kılar.

Ağ	`UPGRADE_ADAM` (ADAM ve Çift Kulvar Başlangıcı)	`UPGRADE_ADAM_V2` (Shift V2)	`UPGRADE_POMBL` (Standart Mod / V12)	`UPGRADE_MODELD` (Model D / PoBLS)	Hedef Blok Süresi
Mainnet	Blok 200 (`nAdamDualTrackHeight = 200`)	Blok 705	Blok 2000	Blok 2200	30 saniye
Testnet	Blok 200 (`nAdamDualTrackHeight = 200`)	Blok 300	Blok 400	Blok 500	30 saniye
Regtest	Blok 200 (`nAdamDualTrackHeight = 1`)	Blok 705	Blok 300	Blok 200	10 saniye

D. Bulmaca Zorluğu Bit Kaydırma (Bit-Shift) Parametreleri

Farklı blok yüksekliklerinde dinamik ve yapılandırılabilir zorluk ölçeklendirmesini korurken, ağ üzerinde bulmaca çözümlerinin "yanlış pozitif seline" (false-positive flood) yol açmasını önlemek için ADAM, `Consensus::Params` (`src/adam.cpp` ve `src/rpc/mining.cpp`) içinde tanımlanan zorluk bit kaydırma çarpanlarını uygular:

- **V1 Zorluk Kaydırması (`nAdamDifficultyShiftV1`)**: Fallback Modunda (`nVersion == 11` veya `Consensus::UPGRADE_MODELD` öncesinde) hedef zorluğu gevşeten bit kaydırma çarpanı. Değer: `10` (Mainnet, Testnet ve Regtest üzerinde).
- **V2 Zorluk Kaydırması (`nAdamDifficultyShiftV2`)**: Standart Modda (`nVersion >= 12`) hem `Consensus::UPGRADE_ADAM_V2` hem de `Consensus::UPGRADE_MODELD` aktif olduğunda hedef zorluğu gevşeten bit kaydırma çarpanı. Değer: `6` (Mainnet, Testnet ve Regtest üzerinde).
- **Etkin Geçiş Yüksekliği**: `nVersion == 11` veya `!NetworkUpgradeActive(nHeight, Consensus::UPGRADE_MODELD)` iken `fFallbackMode` aktif kaldığı için, üretim ortamında `activeShift` değeri Mainnet üzerinde **Blok 2.200**, Testnet üzerinde ise **Blok 500** itibarıyla `10` (`V1`) değerinden `6` (`V2`) değerine geçer (Regtest ortamı ise otomatik testler için `VerifyAdamSolution` içinde hedef sınırını atlar ve `scaledTarget = ~UINT256_ZERO` değerini kullanır).

Mainnet ve Testnet üzerinde bulmaca doğrulaması sırasında `scaledTarget` değeri, konsensüs hedefinin (`Target`) `activeShift` kadar kaydırılıp `powLimit` ile sınırlandırılmasıyla elde edilir:

$$\text{scaledTarget} = \min(\text{Target} \lll \text{activeShift}, \text{powLimit})$$

- `nVersion == 11`, `!NetworkUpgradeActive(nHeight, Consensus::UPGRADE_MODELD)` veya `!NetworkUpgradeActive(nHeight, Consensus::UPGRADE_ADAM_V2)` ise activeShift=nAdamDifficultyShiftV1 (`10`).
- `nVersion >= 12` ve hem `Consensus::UPGRADE_MODELD` hem de `Consensus::UPGRADE_ADAM_V2` aktif ise activeShift=nAdamDifficultyShiftV2 (`6`).

E. Başlangıç Zorluk Limiti (powLimit)

1–199 arasındaki blokların çok hızlı kazılmasını (bu durum çatallanmalara ve kuorum kilitlenmelerine yol açıyordu) önlemek için, başlangıç zorluk hedefi `powLimit` şu şekilde ayarlanmıştır:

powLimit = ($\sim$ UINT256_ZERO) $\gg$ 20

Mainnet ve Testnet üzerinde bu değer tam olarak `1/2^20`'dir (genesis bloğunun `0x1e0ffff0` olan `nBits` değerine eşdeğerdir). Bu, blokların genesis'ten itibaren doğal olarak yaklaşık 30 saniye aralıklarla yerleşmesini sağlayarak düğümlerin (nodes) kararlı P2P bağlantıları kurmasına ve birleşik bir zincir ucunu (chain tip) sürdürmesine olanak tanır.

---

3. Doğrulanabilir Rastgele Fonksiyon (VRF) ve Rolling Seed Yapısı

Madencilerin blok H hash'ini manipüle etmek için işlemleri veya nonce'ları değiştirmesini ve böylece blok $H + 1$ için yapılacak lider seçimini saptırmasını (yani **grinding saldırılarını**) önlemek amacıyla ADAM, bir **Doğrulanabilir Rastgele Fonksiyon (VRF) Rolling Seed** modeli uygular.

Matematiksel Formülasyon

ADAM ağ yükseltmesinin (`Consensus::UPGRADE_ADAM`) aktif olduğu herhangi bir H blok yüksekliği için:

$\text{Seed}_H = \text{Hash}(\text{Seed}_{H-1} \| \text{VRFProof}_{H-1})$

Burada:

- H yüksekliğindeki seçim seed'i (Seed), blok $H + 1$ için liderleri belirlemek üzere kullanılır.
- $H - 1$ yüksekliğindeki VRF kanıtı (VRFProof), $H - 1$ bloğunun seçilen **Koordinatörü** tarafından, hibrit bir BLS12-381 + ECDSA fallback imza mekanizması (`SignBLSWithECDSAFallback`) kullanılarak $H - 2$ yüksekliğindeki önceki seed üzerinde oluşturulan imzadır.
- Bu şema altında Koordinatör, kendi ECDSA gizli anahtarından (private key) deterministik olarak türetilen bir BLS gizli anahtarını kullanarak bir BLS12-381 imzası oluşturur. Yetkilendirmeyi kanıtlamak ve anahtar manipülasyonunu önlemek için Koordinatör, BLS açık anahtarını (public key) kendi ECDSA gizli anahtarını kullanarak imzalar. Doğrulama (`VerifyBLSWithECDSAFallback`) işlemi, hem BLS anahtarının ECDSA yetkilendirmesinin hem de hedef seed'in BLS imzasının geçerli olmasını sağlar.
- Elde edilen imza deterministik olduğundan, Koordinatör'ün belirli bir seed için tam olarak bir geçerli imzası vardır; bu da rolling seed yapısını, Koordinatör imzalayıp yayınlayana kadar tamamen değiştirilemez ve tahmin edilemez kılar.

Düğüm Seçimi (SSLE)

Madencilerin ve koordinatörün seçimi `src/adam.cpp` içindeki `SelectAdamNodes()` tarafından gerçekleştirilir:

1. Aktif düğüm havuzu (kayıtlı Masternode listesi ve Coin-Lock veya PoW-Lock aracılığıyla aktif kayıtlı madenciler) derlenir.

2. Seçim havuzu ağa bağlıdır:

    - **Mainnet & Testnet**: Havuz, aktif Masternode'lardan ve aktif kayıtlı madencilerden dinamik olarak oluşturulur. Bununla birlikte, aktif masternode'lar veya kayıtlar tam olarak kurulmadan önce blok üretiminin kararlı kalmasını sağlamak amacıyla, blok yüksekliği bootstrap limitinin altında olduğu sürece (Mainnet

üzerinde `nAdamBootstrapLimit` = 44.850, Testnet üzerinde 100.000 blok), ağ, blok 1 ila 199 arasındaki blok üreticilerini (coinbase çıktıları) otomatik olarak tarar ve açık anahtarlarını madenci havuzuna ekler. Bu, erken aşamalarda zincirin durmasını (stall) önler.

- **Regtest**: Havuz, otomatik testleri kolaylaştırmak amacıyla harici kayıtları otomatik olarak atlar ve 15 adet deterministik bootstrap açık anahtarı içerir:

  $\text{Pool}_{\text{bootstrap}} = \{\text{DeterministicPubKey}_0, \ldots, \text{DeterministicPubKey}_{14}\}$

3. **Rolling seed'e dayanarak seçim havuzundaki her bir düğüm için benzersiz bir hash sırası (hash rank) hesaplanır**:

   $\text{Rank}_i = \text{Hash}(\text{Seed}_H \| \text{PubKey}_i)$

4. **Havuz,** Rank_i **değerlerine göre artan düzende sıralanır**.

5. **Dual-Track (**8 + 3**) Komite Seçimi (`nAdamDualTrackHeight` = Mainnet/Testnet üzerinde 200, Regtest üzerinde 1; mevcut bir zincirde etkinleştirilirken daima mevcut yüksekliğin ilerisinde bir blok yüksekliğine** ≥ **`UPGRADE_ADAM` ayarlanmalıdır)**: `IsAdamDualTrackActive(H)` aktif olduğunda (`H >= consensus.nAdamDualTrackHeight`), 11 doğrulayıcı koltuğu çift yönlü dinamik taşma (spillover) mekanizmasına sahip `ElectDualTrackValidators()` aracılığıyla iki kulvara ayrılır:

   - **Kulvar 1 — Teminatlı Kulvar (`ADAM_COLLATERALIZED_TRACK_QUOTA = 8` koltuk)**:
     - **Alt-Kulvar 1A (`ADAM_MASTERNODE_TRACK_QUOTA = 7` koltuk)**: **Aktif Masternode'lara (`4.200 KRISTA` teminatlı)** ayrılmıştır. Derleme anında `static_assert(ADAM_MASTERNODE_TRACK_QUOTA >= 7)` ile kilitlenmiştir; böylece Aktif Masternode'lar bootstrap sonrasında da (`GetAdamThreshold = 7`) tek başına kuorum oluşturabilir.
     - **Alt-Kulvar 1B (`ADAM_COIN_LOCK_TRACK_QUOTA = 1` koltuk)**: `250 KRISTA` **Coin-Lock** doğrulayıcılarına ayrılmıştır.
     - Alt-Kulvar 1A ile Alt-Kulvar 1B arasındaki doğrudan kulvar-içi taşma (spillover) sayesinde, havuzda hiç Coin-Lock olmadığında Aktif Masternode'lar Kulvar 1'in 8 koltuğunun tamamını (`7 + 1 = 8`) doldurur; Aktif Masternode sayısı `< 7` olduğunda ise boş koltuklar Açık PoW kulvarına taşmadan önce Coin-Lock doğrulayıcıları tarafından doldurulur.
   - **Kulvar 2 — Açık PoW Kulvarı (`ADAM_OPEN_POW_TRACK_QUOTA = 3` koltuk)**: Açık PoW madencilerine (`10 KRISTA` CLTV kilitli `PoW-Lock`, `Zero-Coin PoW` ve `Blok 1..199` bootstrap anahtarları) ayrılmıştır. Havuzda 3'ten az Açık PoW adayı varsa, Kulvar 2'deki boş koltuklar önce kalan Aktif Masternode'lara, ardından kalan Coin-Lock doğrulayıcılarına devredilir.
   - Bu yapı, 11 doğrulayıcı koltuğunun her koşulda eksiksiz doldurulmasını garanti ederken `4.200 KRISTA` Aktif Masternode'ları korur (`7` doğrulayıcı koltuğu + `1` Koordinatör), `250 KRISTA` Coin-Lock doğrulayıcılarına `10 KRISTA` `PoW-Lock` seyreltmesinden izole `1` koltuklu özel bir teminatlı alt-kulvar sunar ve Açık PoW madencilerini `3` koltuk (`%27,27`) ile sınırlar.
   - **Yedek Doğrulayıcı Koltuk Devri ve Zamana Bağlı Kademeli Eşik Gevşetme (`ResolveAdamCommitteeForTime`, `ValidateAdamCommitteeForTime`, `GetAdamEffectiveThreshold`)**:
     - Bir bloğun üretimi ≥ 60 sn gecikirse (`ADAM_COMMITTEE_FALLBACK_TIMEOUT_SECS = 60`, her `ADAM_COMMITTEE_FALLBACK_SLOT_SECS = 30` saniyelik slotta genişleyerek), 11 kişilik komitede çözüm göndermeyen sessiz koltuklar, bloğun kendi `CCoinsViewCache* pView` görünümüne göre

yalnızca **aynı kulvardaki** (`MN -> MN`, `Coin-Lock -> Coin-Lock`, `Açık PoW -> Açık PoW`) ve o kulvar içindeki sıkı sıralı yedek doğrulayıcılara (`k_t > prevStandbyIdxInTrack[t]` ve `k_t < nReplacedInTrack[t] + nSlot`) devredilebilir.

- Model D (`IsModelDActive`) kapsamında **Eşit Emeğe Eşit Kazanç** prensibi uygulanır (`src/masternode-payments.cpp` içindeki `ComputeModelDValidatorSeatPayments`): 11 kişilik komitedeki her $i \in \{0, \ldots, 10\}$ koltuğu, `vAdamSolutions[i]` içinde geçerli bir PoW çözümü sunduğu sürece — ister asıl birincil doğrulayıcı `vExpectedMiners[i]` tarafından ister asıl koltuk zaman aşımına uğradığında ($t \geq 60$ sn) devreye giren aynı kulvardaki meşru yedek doğrulayıcı `vAdamMiners[i]` tarafından çözülmüş olsun — `%40`'lık Katılımcı Doğrulayıcı ödül havuzundan **kendi kulvarının tam koltuk payını (%100)** (7 Masternode onaylayıcı koltuğu için toplam `%20` [koltuk başı `~%2,857`] + 4 bağımsız madenci koltuğu için toplam `%20` [1 Coin-Lock `%8,00` + 3 Açık PoW koltuk başı `%4,00`]) alır. Dinamik taşma (spillover) sonucu bir Açık PoW madencisi boş bir Coin-Lock koltuğuna yerleştiğinde kendi Açık PoW oranını (`%4,00`, aradaki `%4,00` fark `nUnclaimedPartSplit` içinde kalır), bir Aktif Masternode boş bir Coin-Lock veya Açık PoW koltuğuna yerleştiğinde ise kendi Masternode Doğrulayıcı oranını (`~%2,857`, aradaki `~%5,143` veya `~%1,143` fark sahipsiz kalır) alır. Çözümsüz/boş kalan koltuklar (`vAdamSolutions[i].empty()`) `%0` alırken, Koordinatör yedek doğrulayıcılarla bloğu kurtardığında tam `%15` payını (`nProducerSplit`) korur. Sahipsiz kalan doğrulayıcı payları (`nUnclaimedPartSplit = (nPartSplitTotal - nPaidPartTotal)`; erken bootstrap aşamasında `vLlmqPayees` boş kalırsa `%10` `nLLMQSplit` dahil) **%50 Otomatik Protokol Yakımına (`nExpectedBurn = nUnclaimedPartSplit / 2`)** ve **%50 Geliştirici Hazinesine (`nExpectedTreasury = nTreasurySplit + (nUnclaimedPartSplit - nExpectedBurn)`)** bölüştürülürken Pasif Masternode Kazananı tam `%35` (`nMNSplit`) payını alır. Ayrıca LLMQ blok önerisi doğrulaması (`qblockprop`, yalnızca düğümün mevcut ucu `msg.block.hashPrevBlock` ile eşleştiğinde değerlendirilir) sırasında $t \geq 60$ sn olduğunda, önerilen blokta bir birincil koltuk i yedek anahtarla **değiştirilmişse** (`vAdamMiners[i] != vExpectedMiners[i]`) ve LLMQ üyesinde asıl doğrulayıcı `vExpectedMiners[i]`'ye ait geçerli bir çözüm mevcutsa, LLMQ üyesi öneriyi reddeder ve `QUORUMSIGSHARE` imzalamaz (boş koltuklar `vAdamMiners[i] == vExpectedMiners[i]` P2P yarış koşullarından dolayı dürüst hızlı blokları geciktirmemek adına veto edilmez).
- Blok üretimi ≥ 120 sn (`ADAM_THRESHOLD_RELAXATION_TIMEOUT_SECS = 120`) boyunca tıkanırsa, `GetAdamEffectiveThreshold(nHeight, nTimeElapsed, nActiveMasternodes)` gereken kuorum eşiğini her `60 saniyede` (`ADAM_THRESHOLD_RELAXATION_SLOT_SECS = 60`) `1` azaltarak bootstrap sonrasında (`nHeight >= nAdamThresholdBootstrapLimit`) VEYA havuzdaki aktif Masternode sayısı ≥ 7 olduğunda asgari `ADAM_MIN_RELAXED_THRESHOLD = 5` tabanına (`> 1 Coin-Lock + 3 Açık PoW = 4`, `static_assert` ve çalışma anı `assert` ile kilitli; her zaman en az 1 Aktif Masternode zorunlu), yalnızca bootstrap döneminde aktif Masternode sayısı < 7 iken `ADAM_MIN_RELAXED_THRESHOLD_BOOTSTRAP = 3` tabanına kadar gevşetir. Yalnızca başlık doğrulaması yapan `ContextualCheckBlockHeader` (blok UTXO görünümü henüz mevcut olmadığından senkronizasyon/reorg sırasında yanlış banlamayı önlemek için) `nActiveMasternodes` için `0` geçerken, tam blok doğrulaması (`CheckFormsOfWork`) bloğun kendi `pView` görünümüne göre `GetActiveMasternodePoolCount(nHeight - 1, pView)` değerini kesin olarak uygular.
- Düğüm genelinde sürümden bağımsız ebeveyn-blok koruyucusu (`src/miner.cpp` içindeki `ActiveProposalGuard`), aynı düğümün aynı `(parentHash, coordinatorKey)` için hem PoW hem PoS bloğunu eşzamanlı veya ardışık üretmesini tüm blok sürümlerinde (`v11` ve `v12`) engeller.

6. **Koordinatör Seçimi**:

- **Kural 1 (`Aktif Masternode Sayısı > Eşik`)**: En yüksek sıralamaya sahip Aktif Masternode **Koordinatör** olarak seçilir; kalan havuzdan 11 **Madenci** `ElectDualTrackValidators` ile seçilir.
- **Kural 2 (`Aktif Masternode Sayısı <= Eşik`)**: 11 **Madenci** `ElectDualTrackValidators` ile seçilir ve havuzda seçilmeyen en yüksek sıralamalı anahtar **Koordinatör** olarak atanır.

---

4. Blok Başlığı Uzantıları ve Serileştirme (Serialization)

ADAM ağ yükseltmesi (`Consensus::UPGRADE_ADAM`) aktif olduğunda, bloklar **Versiyon 11** (Fallback Mode) veya **Versiyon 12** (Standart Mod) blok yapıları kullanılarak serileştirilir. `src/primitives/block.h` dosyasındaki `CBlockHeader` sınıfı dört yeni alanla genişletilmiştir:

```
class CBlockHeader {
public:
    // Legacy fields...
    int32_t nVersion;
    uint256 hashPrevBlock;
    uint256 hashMerkleRoot;
    uint32_t nTime;
    uint32_t nBits;
    uint32_t nNonce;

    // ADAM Extended fields (Version >= 11)
    std::vector<CPubKey> vAdamMiners;                  // Public keys of elected miners (and
        coordinator in v11)
    std::vector<std::vector<unsigned char>> vAdamSolutions; // Serialized partial solutions
        (nonce + miner signature)
    std::vector<unsigned char> vAdamVRFProof;          // Coordinator's VRF signature on
        previous seed
    std::vector<unsigned char> vAdamCoordinatorSig;    // Coordinator's signature on final
        block header hash
};
```

4.1 Hashing ve Serileştirme Sıralaması

Nihai blok başlığı hash'ini imzalayan ve `GetHash()` fonksiyonunun dışında tutulan `vAdamCoordinatorSig` alanının aksine, `vAdamVRFProof` blok hash'i hesaplanmadan *önce* üretilir. `GetHash()` sırasında blok başlığı serileştirmesine dahil edilir, bu da VRF kanıtının başlığa kriptografik olarak kilitlenmesini sağlar.

4.2 Durumsuz Blok Hashing Zinciri (Stateless)

Blok hash'ini (`CBlockHeader::GetHash()`) hesaplamak için ADAM, serileştirilmiş başlığı (versiyon 12 ise `vAdamCoordinatorSig` ve LLMQ `vQuorumSig` hariç tutularak), seçilen madencilere karşılık gelen M adet ardışık hashing turundan oluşan bir zincir boyunca hash'ler:

- Fallback modunda (`nVersion == 11`): M = vAdamMiners.size() − 1.
- Standart modda (`nVersion == 12`): M = vAdamMiners.size().

Her bir $i \in \{0, \ldots, M-1\}$ turu için:

1. Önceki blok hash'inden ve tur endeksinden benzersiz bir tur hash'i türetilir:

   $\text{roundHash}_i = \text{Hash}(\text{hashPrevBlock} \| i)$

2. İlk bayt v_i = roundHash$_i$[0] elde edilir.

3. Tek bir ortak asal (coprime) çarpanı m_i hesaplanır:

   $m_i = v_i \mid 1$

   Eğer $m_i < 3$ ise, $m_i = 3$ olarak ayarlanır. Bu, çarpanların 2^{256} değerine göre aralarında asal olmasını sağlayarak %100 entropiyi korur.

4. Tur hashing işlemi gerçekleştirilir:

   - **Fallback Modu (Versiyon 11)** (burada algo$_i$ = Hash(hashPrevBlock‖vAdamMiners[i]) (mod 18)):
     - **Tur 0**: H_0 = CalculateAdamPuzzleHash(algo$_0$, SerializedHeader) $\times m_0 \pmod{2^{256}}$ hesaplanır ve $H_{\text{prev}} = H_0$ atanır.
     - **Tur** $i > 0$: H_i = CalculateAdamPuzzleHash(algo$_i$, H_{prev}) $\times m_i \pmod{2^{256}}$ hesaplanır ve $H_{\text{prev}} = H_i$ olarak güncellenir.
   - **Standart Mod (Versiyon 12)**:
     - **Tur 0**:
       - `GetAdam3PermutationAlgos(hashPrevBlock, vAdamMiners[0])` ile 3-permütasyon algoritmaları algo1, algo2, algo3 türetilir.
       - Hesaplama yapılır:

         $H_0^{(3)}$ = CalculateAdamPuzzleHash(algo3, SerializedHeader)

         $H_0^{(2)}$ = CalculateAdamPuzzleHash(algo2, $(H_0^{(3)} \times 1) \pmod{2^{256}}$)

         H_0 = CalculateAdamPuzzleHash(algo1, $(H_0^{(2)} \times 1) \pmod{2^{256}}$)

       - Çarpan uygulanır:

         $H_{\text{prev}} = (H_0 \times m_0) \pmod{2^{256}}$
     - **Tur** $i > 0$:
       - `GetAdam3PermutationAlgos(hashPrevBlock, vAdamMiners[i])` ile 3-permütasyon algoritmaları algo1, algo2, algo3 türetilir.
       - Hesaplama yapılır:

         $H_i^{(3)}$ = CalculateAdamPuzzleHash(algo3, H_{prev})

         $H_i^{(2)}$ = CalculateAdamPuzzleHash(algo2, $(H_i^{(3)} \times (i + 1)) \pmod{2^{256}}$)

         H_i = CalculateAdamPuzzleHash(algo1, $(H_i^{(2)} \times (i + 1)) \pmod{2^{256}}$)

       - Çarpan uygulanır:

         $H_{\text{prev}} = (H_i \times m_i) \pmod{2^{256}}$

$M - 1$ turunun sonundaki nihai hash H_{prev}, blok hash'i olarak döndürülür.

---

5. Konsensüs Doğrulama Kuralları

Bir blok alındığında, ADAM ağ yükseltmesi (`Consensus::UPGRADE_ADAM`) aktifse, `src/main.cpp` içindeki `CheckBlock()` ve `CheckFormsOfWork()` fonksiyonları aşağıdaki doğrulamaları zorunlu kılar:

1. **Versiyon Zorunluluğu ve Sürüm Düşürme (Downgrade) Koruması**:
   - `Consensus::UPGRADE_ADAM` aktif olduğunda (`nHeight >= 200`), blok versiyonu en az `11` olmalıdır.
   - `Consensus::UPGRADE_POMBL` aktif olduğunda (`vUpgrades[Consensus::UPGRADE_POMBL].nActivationHeight`: Mainnet üzerinde `2.000`, Testnet üzerinde `400`, Regtest üzerinde `300`), hem `ContextualCheckBlockHeader` hem de `CheckBlock` fonksiyonları `block.nVersion >= 12` olmasını zorunlu kılar ve sürüm düşürme saldırılarını önlemek amacıyla `block.nVersion < 12` olan tüm blokları `bad-version` DoS hatası ile reddeder.

2. **Madenciler ve Çözüm Boyutları**:
   - **Versiyon 11 (Fallback Mode)**:
     - `vAdamMiners` boyutu 11 ile 14 arasında (`nAdamThreshold + 1` ile `14` arası) olmalıdır.
     - `vAdamSolutions` boyutu `vAdamMiners.size() - 1` (veya yalnızca ilk `vAdamMiners.size() - 1` komite koltuğu değerlendirilmek üzere `vAdamMiners.size()`) olmalıdır.
     - Doğrulanacak Koordinatör, `vAdamMiners` dizisinin son elemanıdır (`vAdamMiners.back()`).
   - **Versiyon 12 (Standart Mod)**:
     - `vAdamMiners` boyutu tam olarak `nAdamMinersCount` (`11`) değerine eşit olmalıdır.
     - `vAdamSolutions` boyutu tam olarak `vAdamMiners.size()` (`11`) değerine eşit olmalıdır.
     - Koordinatör, `SelectAdamNodes` ($t \geq 45$ sn durumunda `GetAdamFallbackCoordinatorForSlot`) algoritmasından türetilir.

3. **VRF Kanıtı Doğrulaması**: `vAdamVRFProof`, önceki rolling seed Seed_{H-1}'e göre doğrulanmalıdır ve `VerifyBLSWithECDSAFallback` kullanılarak beklenen Koordinatörün açık anahtarıyla eşleşen geçerli bir imza olmalıdır.

4. **Seçim Yolu Doğrulaması (Election Path Validation)**: Mainnet ve Testnet üzerinde hem Versiyon 11 (`vAdamMiners[0..M-1]`) hem de Versiyon 12 (`vAdamMiners[0..10]`) bloklarında komite açık anahtarları `SelectAdamNodes` çıktısıyla (veya $t \geq 60$ sn sonrasında `ValidateAdamCommitteeForTime` ile doğrulanan meşru aynı kulvar yedek devriyle) eşleşmelidir; yalnızca Regtest ortamı kendi kendine yeten birim testleri için Versiyon 11 seçim kontrolünü atlar.

5. **Kısmi Çözümler Doğrulaması (Partial Solutions Validation)**:
   - Geçerli kısmi çözümlerin sayısı `GetAdamEffectiveThreshold(nHeight, nBlockTimeDiff, nActiveMasternodes)` fonksiyonunun belirlediği etkin çoğunluk eşiğini karşılamalıdır (temel eşik olan `GetAdamThreshold(nHeight)`, `nAdamThresholdBootstrapLimit` [Mainnet'te 50.000, Testnet'te 100.000, Regtest'te 1.000] altında `5`, sonrasında ise `nAdamThreshold` [Mainnet/Regtest üzerinde `7`, Testnet üzerinde `3`] olup $t \geq 120$ sn sonrasında Bölüm 3'te açıklandığı şekilde kademeli olarak gevşetilir).
   - Her çözüm bir `nonce` ve bir `signature` (imza) olarak ayrıştırılır.
   - Bulmaca hash'i (puzzle hash), madenciye atanan algoritma(lar) kullanılarak hesaplanır:

- **Versiyon 11 (Fallback Mode)** içinde:

  PuzzleHash = CalculateAdamPuzzleHash(algoIndex, $\text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i$)

  Burada algoIndex = $\text{Hash}(\text{Seed}_H \| \text{MinerPubKey}_i) \pmod{18}$ olup, desteklenen **18 algoritmadan** biri kullanılır (bunlar arasında `Hamsi`, `Fugue`, `Shabal`, `Whirlpool` ve `Haval-256` yer alır).

- **Versiyon 12 (Standart Mod)** içinde: Önceki bloğun hash'ine ve madencinin açık anahtarına dayanarak mevcut 18 algoritma arasından deterministik olarak 3 farklı hashing algoritmasını (algo1, algo2 ve algo3) seçen bir 3-permütasyon seçici şeması GetAdam3PermutationAlgos(hashPrevBlock, MinerPubKey_i) kullanılır. Çözücü bu üç algoritmayı birleştirir:

  $\text{PuzzleHash} = \text{algo1}((\text{minerIdx} + 1) \times \text{algo2}((\text{minerIdx} + 1) \times \text{algo3}(\text{Challenge}))) \pmod{2^{256}}$

  Burada $\text{Challenge} = \text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i$ şeklindedir.

- `PuzzleHash`, `nBits` ile tanımlanan hedef zorluğu karşılamalıdır (Bölüm 2.D'de açıklandığı gibi scaledTarget = Target $\ll$ activeShift olarak gevşetilir).

- İmza, `PuzzleHash`'i imzalayan `MinerPubKey_i`'ye göre doğrulanmalıdır.

6. **Koordinatör İmzası Doğrulaması**: `vAdamCoordinatorSig`, `VerifyBLSWithECDSAFallback` kullanılarak nihai blok başlığı hash'ini (imzanın kendisi hariç) imzalayan beklenen Koordinatörün açık anahtarına göre doğrulanmalıdır.

---

6. Cüzdan ve RPC API Entegrasyonu

Geliştiricileri ve madencilik havuzu operatörlerini desteklemek amacıyla RPC API, işbirlikçi konsensüs alanlarını dışarıya sunar (expose):

`getblocktemplate` İsteği ve Yanıtı

Harici madenci yazılımları (`cpuminer-opt -a adam`, `ccminer -a adam`), cüzdanda kayıtlı belirli bir doğrulayıcı veya Masternode operatör koltuğunu hedeflemek üzere `"mineraddress"` içeren bir JSON şablon isteği (`{"mineraddress": "<KRISTA_adresi>"}`) gönderebilir. `nVersion >= 11` olduğunda `getblocktemplate`, çağrıyı yapan düğümün/adresin mevcut turdaki rolüne göre iki farklı yanıt modu sunar:

- **Harici Komite Bulmaca İş Yanıtı** (yerel cüzdandaki bir doğrulayıcı anahtarı veya belirtilen `"mineraddress"` mevcut turda komiteye seçildiğinde ve bu uç için bulmacasını henüz çözmediğinde döner):
  - `capabilities`, `version` (`11` Fallback Modu veya `12` Standart PoMBL Modu), `previousblockhash`, `transactions` (`[]`), `bits`, `height`, `curtime`
  - `target`: Hex-kodlu 256-bit gevşetilmiş bulmaca hedefi (`scaledTarget = Target << activeShift`).
  - `puzzleheader`: 80-bayt hex-kodlu bulmaca girdi ön eki (`32-bayt adamSeed || 0x21 || 33-bayt sıkıştırılmış myMinerKey || 10-bayt sıfır dolgusu || 4-bayt nonce yuvası`).
  - `powalgo`: Atanan bulmaca algoritması tanımı — Versiyon 11'de `"<algo>"` (örn. `"hamsi"`), Versiyon 12'de ise `"<algo1>+<algo2>+<algo3>+<minerIdx>"` (örn. `"whirlpool+fugue+haval+0"`).

- **Bekleme Yanıtı** (yerel/belirtilen doğrulayıcı anahtarı mevcut turda seçilmediğinde, mevcut uç için bulmacasını zaten çözdüğünde veya aktif havuz boş olduğunda döner — klasik BIP22 şablonuna düşmeden gereksiz CPU/GPU tüketimini önler):
  - `capabilities`, `version`, `previousblockhash`, `transactions` (`[]`), `height`, `curtime` ve `"status": "waiting_for_election"`.
- **Koordinatör Tam Blok Şablonu ve `getblock` JSON Alanları Ayrımı**:
  - `getblocktemplate` tam bir Koordinatör blok şablonu oluşturduğunda (`src/rpc/mining.cpp`), küçük harfli `adamminers` (hex-kodlu sıkıştırılmış açık anahtar dizisi), `adamsolutions`, `adamvrfproof` ve `adamcoordinatorsig` alanlarını döndürür.
  - `getblock` onaylanmış veya aday bir bloğu `blockToJSON` (`src/rpc/blockchain.cpp`) üzerinden serileştirdiğinde ise camelCase `vAdamMiners` (`EncodeDestination(miner.GetID())` ile Base58Check KRISTA adresleri dizisi), `vAdamSolutions`, `vAdamVRFProof` ve `vAdamCoordinatorSig` alanlarını döndürür.

`generate` RPC ve Qt Dashboard Madenci Merkezi (Miner Hub)

`generate` / `setgenerate` RPC komutları ve **KristaTech-Qt Dashboard Madenci Merkezi (Miner Hub)**, ADAM blokları için işbirlikçi bulmaca çözümünü ve blok birleştirme sürecini yönetir. Geleneksel yüksek zorluklu hash yarışması yapmak yerine düğüm:

1. Dönen VRF tohumuna dayanarak yerel cüzdan/Masternode anahtarlarından hangilerinin Komite Doğrulayıcısı veya Koordinatör olarak seçildiğini belirler.
2. Seçilen tüm yerel doğrulayıcı koltukları için hafif kısmi PoW bulmacalarını çözer ve `ADAMSOL` mesajlarıyla ağa yayar.
3. Koordinatör olarak seçildiyse çözümleri toplar, PoW Blok Üreticisi coinbase payını (`vout[0]`) doğrudan seçilen Koordinatörün kendi adresine (`4.200 KRISTA` teminatlı bir Masternode ise soğuk teminat ödeme adresine) yönlendirir, VRF kanıtını ve nihai blok özetini deterministik olarak imzalar (Model D aktif olduğunda 5'te 3 LLMQ `vQuorumSig` sertifikası alır) ve tamamlanmış bloğu ağa sunar. Qt Dashboard Miner Hub ekranı tek tıkla CPU madenciliği, arayüzü dondurmayan tek tıkla `PoW-Lock` / `Coin-Lock` kaydı, canlı komite rolü rozetleri, ayrıştırılmış ödüllü kazanım (🏆) ve çözülen bulmaca (🧩) sayaçları ile maskeli `cpuminer` komutu kopyalama özelliklerini doğrudan masaüstü cüzdan üzerinden sunar.

---

7. Proof-of-Stake (PoS) Entegrasyonu ve İşbirlikçi PoS

ADAM, Proof-of-Stake (PoS) mekanizmasının yerine geçmez, aksine onunla entegre olarak PoS ile PoW'un (ADAM işbirlikçi madenciliği) birlikte çalıştığı bir **Hibrit İşbirlikçi PoS (Hybrid Cooperative PoS)** konsensüs mekanizması oluşturur.

Geleneksel PoS modelinde, blok üretimi yalnızca stake ağırlığına (cüzdanda tutulan coin miktarı) göre belirlenir. ADAM'da bu süreç, blok grinding'i, bencil staking'i ve hedefli lider DoS saldırılarını önlemek için işbirlikçi madenci-koordinatör doğrulama döngüsüyle birleştirilir.

Staking ve İşbirlikçi Yaşam Döngüsü (Blok yüksekliği $\geq$ 200)

Ağ yükseltmesi `Consensus::UPGRADE_POS` etkinleştirildiğinde (Mainnet üzerinde 200. blok yüksekliğinde), blok üretimi saf İşbirlikçi PoW'dan, PoW işbirlikçi madenciliği ile PoS staking'in paralel olarak çalıştığı Hibrit İşbirlikçi PoS modeline geçer:

> **ÖNEMLİ**
>
> **Tek İmzalı PoS Bloklarına İzin Verilmez:** Proof-of-Work (ADAM işbirlikçi madenciliği) asla sonlanmaz veya PoS lehine devre dışı bırakılmaz. KristaTech bünyesindeki her PoS bloğu hibrittir; seçilen ADAM madencileri tarafından çözülen PoW bulmacalarının doğrulanmasını gerektirir ve hem stake edenin anahtarı (`vchBlockSig`) hem de seçilen ADAM koordinatörünün anahtarı (`vAdamCoordinatorSig`) ile çift imzalanmış (double-signed) olması gerekir. Tek imzalı bloklar, konsensüs doğrulama kuralları tarafından kesin bir şekilde reddedilir.

1. **Staking Yetkisi (Kernel Kontrolü)**: Cüzdanın staking iş parçacığı (staking thread - `ThreadStakeMinter`), kernel hash kontrolünü (coin ağırlığı ile orantılı olarak) doğrulayarak herhangi bir UTXO'nun bir blokta stake etmeye uygun olup olmadığını periyodik olarak değerlendirir.

2. **İşbirlikçi Bulmaca Toplama ve Dönüşümlü Slot Açılış Önceliği**: Her 15 saniyelik `TimeProtocolV2` slot açılışında (`src/miner.cpp` içindeki `ADAM_SLOT_PRIORITY_WINDOW_MS = 1500` ms), PoW ve PoS önceliği ebeveyn bloğun türüne göre (`pindexPrev->IsProofOfStake()`) dönüşümlü olarak verilir: önceki blok PoW ise PoS paydaşları beklemesiz (`0 ms`) başlarken PoW koordinatörleri (`BitcoinMiner` ve `TryAssembleCoordinatorBlockAsync`) `1500 ms` bekler; önceki blok PoS ise PoW koordinatörleri beklemesiz (`0 ms`) başlarken PoS paydaşları (`ThreadStakeMinter`) `1500 ms` bekler (öncelikli taraf bu pencerede blok üretmezse diğer taraf aynı 15 saniyelik slot içinde kesintisiz devreye girer). Staking iş parçacığı sırası geldiğinde geçerli bir çekirdek bulduğunda aday blok şablonunu oluşturur (`UPGRADE_POMBL` altında `Versiyon 11` , `UPGRADE_POMBL` ve üzerinde `Versiyon 12`). Paydaş (staker), `SelectAdamNodes` ($t \geq 60$ sn durumunda `ResolveAdamCommitteeForTime`) aracılığıyla seçilen madencileri alır ve `mapAdamSolutionsCache` önbelleğinden kısmi PoW bulmaca çözümlerini toplar; etkin çoğunluk eşiği (`GetAdamEffectiveThreshold`) sağlanana kadar şablonun kesinleştirilmesi ertelenir.

3. **Merkeziyetsiz Koordinatör Doğrulaması ve Ortak İmzalama (`POSCOORDREQ` / `POSCOORDSIG`)**:

   - **Yerel Koordinatör**: Aktif Koordinatör anahtarı ($t < 45$ sn için birincil `expectedCoordinator` , $t \geq 45$ sn için `GetAdamFallbackCoordinatorForSlot`) paydaşın kendi düğümündeyse, düğüm `AcquireCoordinatorProposalSlot` yuvasını ayırır ve `vAdamVRFProof` ile `vAdamCoordinatorSig` alanlarını doğrudan imzalar.
   - **Uzak Koordinatör (P2P Ortak İmzalama)**: Aktif Koordinatör ayrı bir Masternode veya doğrulayıcı düğümdeyse, paydaş `ComputePoSCoordRequestHash` özetini coinstake girdisinin ECDSA anahtarıyla imzalar (`SignPoSCoordRequest`) ve P2P ağı üzerinden çok sekmeli (multi-hop) yayılan bir `POSCOORDREQ` mesajı gönderir. Eş düğümler iletim öncesinde `CheckBlockTime` , `CheckBlock(..., fCheckSig=false)` , `ContextualCheckBlock` , `CheckProofOfStake` ve `ConnectBlock(..., fJustCheck=true)` kontrollerini yürütür; aktif Sıcak Koordinatör ise ek olarak `VerifyPoSCoordRequestAuth` ve `TestBlockValidity` kontrollerini doğrular, aynı `(parentHash, coordinatorKey)` için çift PoW+PoS

üretimini engelleyen `AcquireCoordinatorProposalSlot` yuvasını kilitler, `vAdamVRFProof` + `vAdamCoordinatorSig` imzalarını üretir ve P2P üzerinden `POSCOORDSIG` yanıtını yayınlar. Model D aktif olduğunda paydaş ardından 5'te 3 LLMQ `vQuorumSig` sertifikasını (`RequestBlockQuorumSig`) talep eder.

4. **Stake Eden Blok İmzası (Çift Kilitleme)**: Tamamlanan blok, iki ayrı imza türü kullanılarak güvence altına alınır:
   - **ADAM İmzası (`vAdamCoordinatorSig`)**: İşbirlikçi PoW turunun ve VRF tohum geçişinin geçerliliğini onaylamak üzere aktif Koordinatör tarafından (yerel olarak veya `POSCOORDSIG` aracılığıyla) oluşturulur.
   - **PoS Blok İmzası (`vchBlockSig`)**: Stake edenin cüzdanı tarafından `SignBlock` kullanılarak oluşturulur (stake edilen UTXO'nun gizli anahtarı kullanılarak nihai blok hash'i imzalanır).

Ağ Üzerinde Çift Doğrulama (Dual Validation)

Bir eş düğüm (peer) İşbirlikçi bir PoS bloğu aldığında, `CheckBlock` içindeki doğrulama kuralları her iki konsensüs kontrolünün de geçmesini zorunlu kılar:

1. **Proof-of-Stake Doğrulaması**: Düğüm, `coinstake` işlemini doğrular, kernel hash hedef zorluğunu kontrol eder ve stake edenin blok imzasını (`vchBlockSig`) doğrular.
2. **ADAM İşbirlikçi PoW Doğrulaması**: Düğüm, seçilen madencilerin kısmi PoW bulmaca çözümlerini (T kuorum eşiğini karşılayan `vAdamSolutions`), Koordinatörün VRF kanıtını (`vAdamVRFProof`), Koordinatörün imzasını (`vAdamCoordinatorSig`) ve Model D aktif olduğunda Versiyon 12 bloklarındaki LLMQ kuorum imzasını (`vQuorumSig`) doğrular.

---

8. LLMQ (Long-Living Masternode Quorums) ve DKG (Distributed Key Generation)

Standart Mod altında Versiyon 12 blok şablonlarını doğrulamak ve imzalamak için ağ, **Dağıtık Anahtar Üretimi (DKG - Distributed Key Generation)** oturumlarını yürüten **Uzun Ömürlü Masternode Kuorumlarına (LLMQ - Long-Living Masternode Quorums)** dayanır.

> **NOT**
>
> **Eşik Değerleri Arasındaki Fark: ADAM İşbirlikçi Madencilik Eşiği** (madenci bulmaca çözümleri `vAdamSolutions` için uygulanan 11'de 7 kuralı) ile **LLMQ İmza Doğrulama Eşiği** (blok imzası `vQuorumSig` için uygulanan 5'te 3 kuralı) karıştırılmamalıdır:
>
> - **ADAM Eşiği**: 11 seçilmiş madenciden en az 7'sinin (`nAdamThreshold = 7` Mainnet/Regtest üzerinde, `3` Testnet üzerinde; `nAdamThresholdBootstrapLimit` altındaki başlangıç aşamasında ise `5`) PoW bulmaca çözümünü (`vAdamSolutions`) çözmüş ve göndermiş olmasını şart koşar. Bu kural ADAM işbirlikçi bulmaca doğrulama katmanına aittir.
> - **LLMQ Eşiği**: Blok hash'inin geçerli sayılabilmesi için 5 LLMQ üyesinden en az 3'ünün (`quorum.members.size() / 2 + 1`) deterministik BLS12-381 anahtarlarıyla (`DeriveBLSFromCKey` ; `secp256k1` kimlik anahtarlarıyla yetkilendirilmiş ve `CQuorumSignature` üzerinden `vQuorumSig` içine toplanmış olarak) bloğu imzalamış olmasını şart koşar. Bu kural merkeziyetsiz blok doğrulama katmanına aittir.

A. Kuorum Topolojisi ve Parametreleri

- **Kuorum Boyutu**: Her aktif LLMQ tam olarak **5 üyeden** oluşur (`llmq.cpp:218`).
- **DKG Aralıkları ve Blok Bazlı Rotasyon**:
  - **Başlangıç Epoch Modu**: Mainnet'te her **100 blokta** bir ve Testnet/Regtest'te her **10 blokta** bir yeni bir DKG oturumu gerçekleştirilir (`GetActiveQuorum`).
  - **Blok Bazlı Kuorum Rotasyonu**: Kuorum rotasyonu aktifleştiğinde (Mainnet'te **1.400**, Testnet'te **1.000** ve Regtest'te **800** blok yüksekliğinde), `GetActiveQuorum` her blok yüksekliğinde `((nHeight / dkgInterval) * dkgInterval) - 1` yüksekliğindeki epoch tohumunu kullanarak `RunDKG(nHeight)` çalıştırır.
- **Kuorum İmzası Doğrulama Eşiği (`CQuorumSignature::Verify`)**:
  - **Tüm Ağlar (Mainnet, Testnet, Regtest)**: Model D (`UPGRADE_MODELD`) aktif olduğunda, `CQuorumSignature::Verify` fonksiyonu BFT çoğunluk eşiğini (`quorum.members.size() / 2 + 1` , en az 2 ve en fazla kuorum boyutu olacak şekilde) zorunlu kılar; bu da 5 üyeli standart bir kuorum için en az **5'te 3** imza gerektirir.
  - **Model D Aktivasyonundan Önce**: Mainnet (2.000–2.199 blokları) ve Testnet (400–499 blokları) üzerinde başlangıç masternode birikimine olanak tanımak amacıyla kuorum imzası doğrulaması `CheckBlock()` içinde (`fFallbackMode`) atlanır. Regtest üzerinde Model D yüksekliği 200'de (`UPGRADE_POMBL` yüksekliği 300'den önce) aktifleştiği için 5'te 3 çoğunluk eşiği 300. bloktan itibaren her zaman aktiftir.

B. Aktif Masternode Filtreleme (%100 Zincir Üstü Deterministik Havuz)

- Kuorumlar ve ADAM doğrulayıcı seçimleri, yerel P2P bellek listesine (`mnodeman`) bağımlı olmaksızın %100 zincir üstü (on-chain) doğrulanmış Masternode durumundan (`mapMasternodeCollateralOutpoints` , `mapOnChainCollateralDelegations` ve `pcoinsTip` üzerindeki 4.200 KRISTA teminat doğrulaması) deterministik olarak seçilir.

- 5’ten az zincir üstü aktif masternode kayıtlıysa ağ, kuorum üyelerini deterministik olarak kayıtlı madenci havuzundan (`GetAdamMinerPool(nHeight - 1)` — blok 1-199 bootstrap madenciliği veya coin-lock/pow-lock kayıtları) seçmeye geri döner.
- Mainnet, Testnet veya Regtest üzerinde kod içinde sabitlenmiş (hardcoded) herhangi bir anahtar kimliği (key ID) filtrelemesi veya yerel cüzdan kısıtlaması uygulanmaz, bu da tamamen merkeziyetsiz ve güven gerektirmeyen (trustless) bir konsensüs ortamı sağlar.

C. Deterministik Epoch-Tohum Hash Sıralaması ve Blok Bazlı Canlılık Denetimi

Tüm düğümlerin kuorum üyeliği üzerinde %100 mutabakata varmasını sağlarken kuorum üyelerini hedefli DoS/DDoS saldırılarına karşı korumak amacıyla (`src/llmq.cpp` içindeki `ElectQuorumMembers`):

- Zincir üstü aktif her Masternode (`IsMasternodeActiveAtHeight(..., 2880)` ve `IsValidMasternodeCollateralAndAuth` ile doğrulanır), dönen ADAM tohumu ve kendi teminat UTXO çıktısı kullanılarak deterministik olarak puanlanır:

  $\text{Score}_i = \text{SHA256D}(\text{Seed}_{\text{seedHeight}} \| \text{collateralTxid}_i \| \text{collateralVout}_i)$

- Adaylar Score_i değerine göre artan sırada dizilir ve **ilk 5** aday aktif kuorumu (`nQuorumSize = 5`) oluşturur. Zincir üstü aktif Masternode sayısı 5’in altındaysa, `ElectQuorumMembers` aday listesini deterministik olarak `GetAdamMinerPool(nHeight - 1, pView)` havuzundan doldurur.

- **Epoch Tohumu ve Blok Bazlı Canlılık**: Rotasyon aktivasyonu öncesinde `GetActiveQuorum`, `nDkgHeight = max(vUpgrades[UPGRADE_POMBL].nActivationHeight, (nHeight / dkgInterval) * dkgInterval)` yüksekliğinde `seedHeight = nDkgHeight - 1` kullanarak `RunDKG(nDkgHeight)` çalıştırır. Epoch-tohum rotasyonu aktifleştiğinde ise `GetActiveQuorum`, `seedHeight = ((nHeight / dkgInterval) * dkgInterval) - 1` (`dkgInterval` Mainnet üzerinde `100`, Testnet/Regtest üzerinde `10`) yüksekliğindeki deterministik epoch tohumunu kullanarak zincir üstü Masternode canlılığını **her blok yüksekliğinde** (`RunDKG(nHeight)`) yeniden denetler; böylece dönem boyunca kararlı DKG kimliği korunurken teminatı harcanan veya 2.880 blokluk heartbeat süresi dolan düğümler anında rotasyon dışına çıkarılır:
  - **Regtest**: Blok yüksekliği ≥ 800
  - **Testnet**: Blok yüksekliği ≥ 1.000
  - **Mainnet**: Blok yüksekliği ≥ 1.400

---

9. Çevrimdışı Koordinatör Geri Çekilme Rotasyonu (`GetAdamFallbackCoordinatorForSlot`)

Birincil seçilen Koordinatörün çevrimdışı olması durumunda **hem** Fallback Modunda (Blok Versiyonu 11) **hem de** Standart Modda (Blok Versiyonu 12; Model D öncesi `2.000..2.199` blokları ve Model D ≥ 2.200 blokları dahil) zincirin kilitlenmesini önlemek amacıyla:

A. Birincil Koordinatör Münhasırlık Penceresi ($t < 45$ sn)

Ebeveyn bloğun zaman damgasından itibaren geçen ilk **45 saniye** boyunca (`nBlockTimeDiff < ADAM_FALLBACK_COORDINATOR_TIMEOUT_SECS = 45`):

- **Versiyon 12 (`nHeight >= UPGRADE_POMBL`)**: Hem `CheckBlock` (`src/main.cpp`) hem de blok üretimi (`src/miner.cpp`), `vAdamVRFProof` ve `vAdamCoordinatorSig` alanlarını **yalnızca** `SelectAdamNodes` tarafından seçilen birincil `expectedCoordinator` anahtarının imzalamasını zorunlu kılar. $t < 45$ sn içinde yapılan erken yedek rotasyon denemeleri Koordinatör imzası doğrulamasından geçemez ve reddedilir (`bad-adam-coord-sig`).
- **Versiyon 11 (`UPGRADE_ADAM <= nHeight < UPGRADE_POMBL`)**: Madenciler ve paydaşlar `vAdamMiners.back()` anahtarını döndürmeden önce 45 saniyelik birincil pencereyi bekler; `CheckBlock` ise `vAdamMiners.back()` konumundaki kendi kendine yeten Koordinatör anahtarını doğrular.

B. PoBLS XOR Sıralamalı Yedek Rotasyon ($t \geq 45$ sn)

`nBlockTimeDiff >= 45` saniye olduğunda, koordinatörlük yetkisi geçerli bulmaca çözümü sunmuş komite doğrulayıcıları arasında **30 saniyelik dilimlerle** döndürülür (`src/adam.cpp` içindeki `GetAdamFallbackCoordinatorForSlot`):

1. Bulmacasını çözmüş komite doğrulayıcıları PoBLS XOR mesafelerine göre $D_i = |T_i \oplus T_{\text{target}}|$ artan sırada deterministik olarak sıralanır (`RankValidatorsByPoBLSDistance`; bilet verisi yoksa komite sırası kullanılır).
2. `nRawSlot = (nBlockTimeDiff - 45) / 30` dilimi (`nSlotOffset = (nBlockTimeDiff - 45) % 30`) için aktif yedek Koordinatör (`activeFallback`) sıralı doğrulayıcı listesi üzerinden modüler olarak seçilir; `CheckBlock` ise 30 saniyelik slot geçişlerindeki küçük P2P yayılım gecikmelerini tolere etmek amacıyla `expectedCoordinator`, `activeFallback` veya 5 saniyelik sınır tolerans penceresi içinde (`nRawSlot > 0 && nSlotOffset < 5`) bir önceki dilimin `prevFallback` anahtarını geçerli kabul eder.
3. Versiyon 12 bloklarında `vAdamMiners` her zaman tam `nAdamMinersCount = 11` komite doğrulayıcısı düzenini korur (`vAdamSolutions.size() == 11`); yedek Koordinatör `vAdamMiners` dizisinin boyutunu değiştirmeden doğrudan `vAdamVRFProof` ve `vAdamCoordinatorSig` alanlarını imzalar.

---

10. Konsensüs Kesinliği, Çatal Çözümü (Fork Resolution) ve Performans İyileştirmeleri (v1.4.5.9)

A. Deterministik Eşit İş (Equal-Work) Çatal Çözücü (`CBlockIndexWorkComparator`)

30 saniyelik işbirlikçi PoW/PoS hibrit zincirinde, ağ bölünmeleri veya eşzamanlı geri çekilme rotasyonları aynı yükseklikte ve eşit kümülatif zincir işine (`nChainWork`) sahip rakip bloklar üretebilir. Önceden `src/chain.h` içindeki `CBlockIndexWorkComparator`, blok özetlerini karşılaştırmadan önce yerel geliş sırasına (`nSequenceId`) bakıyordu; bu da bölünmüş alt ağlardaki düğümlerin bir sonraki blok kazılana kadar farklı zincir uçlarında kalmasına yol açabiliyordu.

- **Kural**: `pa->nChainWork == pb->nChainWork` olduğunda, karşılaştırıcı `nSequenceId` kontrolünden önce sayısal olarak daha küçük blok özetine (`pa->GetBlockHash() < pb->GetBlockHash()`) sahip blok indeksini deterministik olarak tercih eder.
- **Etki**: Ağ bölünmesi iyileştiğinde ve rakip bloklar yayıldığında, tüm düğümler bir sonraki bloğu beklemeden anında (`< 1s` içinde) aynı kanonik zincir ucunda birleşir.

B. LLMQ Çifte İmza Önleme (Anti-Equivocation) ve Çok Sekmeli (Multi-Hop) Gossip İletimi

- **Ebeveyn Blok Bazlı Çifte İmza Koruması (`llmq::AllowQuorumSignature` / `mapSignedProposalByPrev`)**: `src/llmq.cpp` içinde her LLMQ kurul üyesi `mapSignedProposalByPrev[prevHash]` kaydını tutar. Bir kurul üyesi belirli bir `prevHash` üzerinde geçerli bir `PROPOSEBLOCK` imzaladığında, `QUORUM_EQUIVOCATION_GUARD_SECS = 30` saniye boyunca rakip koordinatörlerin alternatif blok önerilerini imzalamayı reddeder (yayılım aksaklığı durumunda aynı koordinatöre `12` saniye sonra yeniden deneme izni tanır). Bu sayede aynı 30 saniyelik dilimdeki iki rakip bloğun ikisinin birden 5'te 3 korum sertifikası (`vQuorumSig`) alması engellenir.
- **Çok Sekmeli Gossip İletimi (`setSeenProposedBlocks` , `mapQuorumBlockSigs`)**: `PROPOSEBLOCK` (`qblockprop`) ve `QUORUMSIGSHARE` (`qsigshare`) P2P mesajları tekilleştirilerek tüm bağlı eşlere iletilir; böylece doğrudan (tek sekmede) bağlı olmayan korum üyeleri ve blok önericileri de öneri ve imza paylarını eksiksiz alır.
- **`ElectQuorumMembers` İçinde %100 Zincir Üstü (On-Chain) Yedek Havuz**: Son 2.880 blok içinde zincir üstü heartbeat ping göndermiş ve harcanmamış 4.200 KRISTA teminatına sahip Masternode sayısı `nQuorumSize` (5) değerinin altındaysa, `ElectQuorumMembers` fonksiyonu bellek içi P2P `mnodeman` durumuna başvurmadan doğrudan zincir üstü `GetAdamMinerPool(nHeight - 1, pView)` aday havuzuna deterministik olarak geri döner.

C. Doğrulanmış Zincir Üstü Masternode Heartbeat Ping Desteği (`ExtractOnChainMasternodePing` & `SendOnChainPing`)

- **Sıkıştırılmış (33-bayt) ve Sıkıştırılmamış (65-bayt) Açık Anahtar Desteği**: `src/adam.cpp` içindeki `ExtractOnChainMasternodePing` , serileştirilmiş `CDataStream` `OP_RETURN` kanıtıyla (`collateralOutpoint` + `Hash(collateralOutpoint || tx.vin[0].prevout)` üzerindeki `vchPingSig` imzası + isteğe bağlı `bool hasDelegation` + `COnChainMasternodeDelegation`) birlikte hem 36 baytlık (`0x21 <33-bayt sıkıştırılmış pubkey> OP_CHECKSIGVERIFY OP_1`) hem de 68 baytlık (`0x41 <65-bayt sıkıştırılmamış pubkey> OP_CHECKSIGVERIFY OP_1`) MESCAL `check-signature-verification` heartbeat betiklerini doğrular.
- **İletim Ücreti Muafiyeti (`0.0001 KRISTA`)**: Kimliği doğrulanmış zincir üstü Masternode ping işlemleri `GetMinRelayFee` (`src/main.cpp`) içindeki `1000x` `OP_RETURN` ücret çarpanından muaftır ve sabit `0.0001 KRISTA` işlem ücretiyle yayınlanır.

D. Kilitsiz (Lock-Free) Zincir Ucu Erişimi (`CChain::AtomicTip()`)

- `CChain` (`src/chain.h` / `src/chain.cpp`), `CChain::SetTip()` içinde güncellenen atomik bir `std::atomic<CBlockIndex*> pAtomicTip` işaretçisi tutar.
- `GetChainTip()` (`src/main.cpp`), `cs_main` kilidini almadan doğrudan `chainActive.AtomicTip()` değerini okur; böylece P2P mesaj işleme, RPC sorguları ve staking/madencilik iş parçacıkları arasındaki kilit çekişmesi (lock contention) ortadan kaldırılır.

E. Merkeziyetsiz PoS Ortak İmzalama, Kriptografik Gelecek Kuyruğu ve Tek Dağıtıcılı Çözücü (`Rev 40–42`)

- **Merkeziyetsiz `POSCOORDREQ` / `POSCOORDSIG` P2P Ortak İmzalama (`K4` / `R42-F2`)**: Aktif Koordinatör uzak bir düğümdeyken PoS paydaşlarının P2P üzerinden `vAdamVRFProof` ve `vAdamCoordinatorSig` imzalarını güven gerektirmeden almasını sağlar (`VerifyPoSCoordRequestAuth` , `TestBlockValidity` ,

`AcquireCoordinatorProposalSlot` çift PoW+PoS üretim koruması ve `POSCOORDSIG` alındığında `mapSeenPoSCoordReqs` üzerindeki `targetCoordinator`, `draftHeader` ve ebeveyn `adamSeed` ile önbellekleme/aktarım öncesi tam `VerifyAdamVRFProof` + `VerifyAdamCoordinatorSig` doğrulaması).

- **Sınırlandırılmış `mapFutureBlocks` Kriptografik Ön-Doğrulaması (`K1` / `R42-F1`)**: `IsFutureBlockPreVerified(header, pindexPrev)`, `mapBlockIndex` içinde bilinen bir ebeveyn (`header.GetBlockTime() > pindexPrev->MinPastBlockTime()`) zorunlu kılar; `nVersion >= 11` için `VerifySelfContainedBLSSignedData` (`vrfBlsPk == coordBlsPk`) ve `GetAdamMinerPool` doğrulayıcı sahipliğini (`VerifyAdamVRFProof` + `VerifyAdamCoordinatorSig`), `nVersion >= 12` ve `IsModelDActive` durumunda ise aktif LLMQ kurulu üzerinde 5'te 3 `CQuorumSignature::Verify` imzasını doğrular; ayrıca kuyruktaki tam `CBlock` kayıtlarının `HEADERS` mesajlarıyla ezilmesini önler ve `AddInventoryKnown` çağrısını `ProcessFutureBlocks` aşamasına erteler.
- **Tek Dağıtıcılı Strided Çözücü, Deterministik `DeriveBLSFromCKey` ve Yaşam Döngüsü Korumalı Koordinatör Birleştirme (`O2`–`O5` / `R42-F3`)**: Çok çekirdekli `BitcoinMiner` bulmaca aramasını mükerrer nonce taraması olmadan koordine eder, `AutoRegisterMiner` ve `registerminer` yollarını ortak `CMinerRegistrationSlotGuard` ve deterministik `DeriveBLSFromCKey` ile birleştirir, `TryAssembleCoordinatorBlockAsync` iş parçacığını `GenerateBitcoins` geçişlerinde ve `g_connman.reset()` sonrası kapanışta güvenli biçimde sonlandırır ve `ccminer` tarafına Versiyon 12 3-permütasyonlu bileşik bulmaca desteğini (`<a1>+<a2>+<a3>+<minerIdx>`) ekler.